

Task 1:

- Python version: Python 3.13.11
- how to create the environment: `virtualenv taskenv(envname)` or `python -m venv taskenv(envname)`
- how to activate it: `taskenv(envname)/Scripts/Activate`
- how to install dependencies: `pip install -r requirements.txt`
- how to run the project: `python manage.py runserver`
- how to run tests: For Single File: `python -m unittest tests.test_analyzer -v`, For all Files: `python -m unittest discover -s tests -p "test_*.py" -v`
- how to run Employee Analyzer: `python -m src.employee_analyzer.main`
- how to create requirements.txt: `pip freeze > requirements.txt`

Employee Management System – R&D Document

1. Problem Understanding

The goal of this project is to create a simple **Employee Management System using Python**.

The user should be able to:

- Add an employee
- Remove an employee
- Update an employee
- Find an employee by ID
- List employees
- Search employees by name
- Save employee data in a JSON file
- Load employee data from a JSON file
- Validate employee information
- Handle errors properly
- Keep application logs

The project is divided into three files:

- validation.py – takes employee ID and name from the user and validates them.
 - employee.py – contains employee validation and employee operations.
 - main.py – controls the application, menu, JSON file handling, logging, and error handling.
-

2. Assumptions

The following assumptions are used in the project:

1. Every employee has an ID and a name.
 2. Employee ID must be an integer.
 3. Employee ID must be greater than 0.
 4. Employee name cannot be empty.
 5. Spaces-only names are also not allowed.
 6. Employee IDs should be unique.
 7. Employees are stored in a list while the program is running.
 8. Each employee is stored as a dictionary.
 9. Employee data is saved in employees.json.
 10. Application logs are stored inside the logs folder.
 11. If employees.json does not exist, the application starts with an empty list.
 12. Invalid data from the JSON file is not accepted.
 13. The application continues running until the user selects Exit or stops the program using Ctrl+C.
-

3. Data Structure

The project uses a **list of dictionaries** to store employees.

The employee list starts as:

```
employees = []
```

One employee looks like:

```
{  
    "id": 101,
```

```
"name": "Rahul"
}
```

Multiple employees are stored like:

```
employees = [
  {
    "id": 101,
    "name": "Rahul"
  },
  {
    "id": 102,
    "name": "Amit"
  }
]
```

Why List?

A list is used because we need to store multiple employees.

It also makes it easy to:

- Add employees
- Remove employees
- Loop through employees
- Search employees
- Display employees
- Load data from JSON

Why Dictionary?

A dictionary is used for one employee because it allows us to store the employee's information using clear keys:

```
employee["id"]
```

```
employee["name"]
```

This makes the data easy to understand.

4. Program Flow

The application starts from the main() function.

The basic flow is:

Start

↓

Setup Logging

↓

Load Employee Data

↓

Display Menu

↓

Get User Choice

↓

Perform Selected Operation

↓

Save Data if Required

↓

Show Menu Again

↓

Exit

Application Flow in Detail

Step 1 – Setup Logging

The application first runs:

```
setup_logging()
```

It checks whether the logs folder exists.

If the folder does not exist, it creates it.

The log file is:

logs/application.log

Step 2 – Load Employee Data

The application calls:

load_data()

It checks whether employees.json exists.

If the file does not exist, the application starts with an empty employee list.

If the file exists:

1. JSON data is loaded.
 2. It checks that the data is a list.
 3. Each employee is validated.
 4. Valid data is loaded into the employees list.
-

Step 3 – Display Menu

The user gets the following options:

1. Add Employee
 2. Remove Employee
 3. Update Employee
 4. Find Employee
 5. List Employees
 6. Search Employees
 7. Exit
-

Step 4 – Get User Choice

The user's choice is converted into an integer.

If the user enters something like:

abc

a ValueError is handled and the user is asked to enter a number.

Step 5 – Perform Operation

Based on the user's choice, the application calls the required function.

For example:

`add_employee()`

`remove_employee()`

`update_employee()`

`find_employee()`

`list_employees()`

`search_employees()`

Step 6 – Save Data

After adding, removing, or updating an employee, the application calls:

`save_data()`

The data is also saved when the user selects Exit.

5. Validation Rules

Validation is done in two places.

A. User Input Validation

This is handled in `validation.py`.

Employee ID

The ID:

- Must be a number.
- Must be an integer.
- Must be greater than 0.

Valid:

101

Invalid:

0

-10

abc

If the user enters a non-numeric value, ValueError is handled.

Employee Name

The name:

- Cannot be empty.
- Cannot contain only spaces.
- Leading and trailing spaces are removed using `.strip()`.

Valid:

Rahul

Invalid:

""

" "

B. Employee Record Validation

The `is_valid_employee()` function checks whether an employee record is valid.

A valid employee must:

- Be a dictionary.
- Have an id.
- Have a name.
- Have an integer ID.
- Have an ID greater than 0.
- Have a string name.
- Have a non-empty name.

Example:

```
{
```

```
"id": 101,  
"name": "Rahul"  
}
```

6. Employee Operations

Add Employee

The application:

1. Gets the employee ID.
2. Gets the employee name.
3. Checks whether the ID already exists.
4. Creates an employee dictionary.
5. Adds it to the employee list.
6. Saves the data.

If the ID already exists, the employee is not added.

Remove Employee

The application:

1. Gets the employee ID.
2. Searches for the employee.
3. Removes the employee if found.
4. Saves the updated data.

If the employee does not exist, it shows:

Employee not found.

Update Employee

The application:

1. Gets the employee ID.
2. Finds the employee.

3. Gets the new name.
4. Updates the name.
5. Saves the updated data.

The employee ID is not changed.

Find Employee

The user enters an employee ID.

The application uses:

`find_employee_by_id()`

to find the employee.

If found, the employee ID and name are displayed.

If not found:

Employee not found.

is displayed.

List Employees

The application checks the employee list and displays valid employees.

If there are no valid employees:

No employees available.

is displayed.

Search Employees

The user enters a name to search.

The search is **case-insensitive** and supports partial matching.

For example, searching:

rah

can find:

Rahul

because the code compares the lowercase versions of both names.

If there is no match:

No matching employee found.

is displayed.

7. JSON File Handling

The application uses:

```
DATA_FILE = "employees.json"
```

to store employee data permanently.

Loading Data

The `load_data()` function handles:

- JSON file not found
- Invalid JSON
- JSON data that is not a list
- Invalid employee records
- File reading errors

Example JSON data:

```
[
  {
    "id": 101,
    "name": "Rahul"
  },
  {
    "id": 102,
    "name": "Amit"
  }
]
```

Saving Data

The `save_data()` function saves the employee list using:

```
json.dump(employees, file, indent=4)
```

8. Logging and Error Handling

The project uses Python's logging module.

The logger name is:

```
employee_management
```

Logs are saved in:

```
logs/application.log
```

Three main logging levels are used.

INFO

Used for normal successful operations.

Examples:

- Employee added
- Employee removed
- Employee updated
- Employee found
- Data loaded
- Data saved
- Application started
- Application exited

WARNING

Used for invalid input or expected problems.

Examples:

- Invalid employee ID
- Invalid employee name
- Duplicate employee ID
- Employee not found

- Invalid menu choice
- No search result

ERROR

Used for more serious data or file problems.

Examples:

- Invalid employee data
- Invalid JSON
- File read error
- File save error

Exception Handling

The application handles:

- ValueError
- JSONDecodeError
- OSError
- KeyboardInterrupt
- Unexpected exceptions

This prevents common errors from suddenly stopping the application.

9. Possible Edge Cases

Some possible problems that the application handles are:

1. User enters a non-numeric employee ID.
2. User enters 0 as an employee ID.
3. User enters a negative employee ID.
4. User enters an empty employee name.
5. User enters only spaces as the employee name.
6. User tries to add an existing employee ID.
7. User tries to find an employee that does not exist.
8. User tries to update an employee that does not exist.

9. User tries to remove an employee that does not exist.
 10. There are no employees in the list.
 11. No employee matches a search.
 12. employees.json does not exist.
 13. employees.json contains invalid JSON.
 14. JSON data is not stored as a list.
 15. JSON contains an invalid employee record.
 16. The application cannot read or write the JSON file.
 17. The user stops the application using Ctrl+C.
-

10. Chosen Approach

The project uses a **simple modular approach**.

The code is divided into three files:

employee_management/

|

├— validation.py

├— employee.py

└— main.py

validation.py

Responsible for user input validation.

get_employee_id()

get_employee_name()

employee.py

Responsible for employee-related operations.

is_valid_employee()

find_employee_by_id()

get_valid_employees()

add_employee()

remove_employee()

update_employee()

find_employee()

list_employees()

search_employees()

main.py

Responsible for application-level operations.

setup_logging()

load_data()

save_data()

display_menu()

get_menu_choice()

handle_menu_choice()

run_application()

main()

This keeps each part of the application focused on its own job.

11. Alternative Approaches Considered

Approach 1 – Put Everything in One File

All validation, employee operations, menu handling, JSON handling, and logging could be written in one file.

Problem

The file would become large and difficult to understand.

It would also be harder to test and maintain.

Approach 2 – Use Only a List

The application could store employees only in:

employees = []

Problem

All data would be lost when the program closes.

There would be no permanent employee storage.

Approach 3 – Use a Database

SQLite could be used instead of JSON.

Advantage

A database is better when the application has more data and more complex requirements.

Problem

For this project, a database would add extra complexity that is not required.

12. Why the Selected Approach Was Chosen

The current approach is simple and fits the requirements of the project.

A **list of dictionaries** is easy to understand and works well for storing employee records.

JSON is used so that employee data is not lost when the application closes.

The code is separated into different files so that validation, employee operations, and application control are not mixed together.

Logging and exception handling are also included to make the application easier to debug and safer when errors occur.

The main goal is to keep the code **simple, readable, and easy to maintain**, instead of trying to make the code shorter or unnecessarily advanced.

Task 8 — Functions and Clean Code

Objective

The purpose of Task 8 was to improve the structure of the application by giving each function a clear responsibility.

The focus was on:

- Clear function responsibilities

- Meaningful names
- Simple parameters
- Useful return values
- Reusable functions
- Avoiding repeated code
- Readable code
- Separation of responsibilities

The goal was **not to reduce the number of lines**. The goal was to make the code easier for another developer to understand.

13. Function Responsibility

Each function should have one clear job.

For example:

```
def get_employee_id():
```

gets and validates an employee ID.

```
def find_employee_by_id(employee_id):
```

finds an employee using the ID.

```
def save_data():
```

saves employee data to the JSON file.

Because each function has a specific job, it is easier to understand and test the code.

14. Function Naming

Meaningful names are used throughout the project.

Examples:

```
get_employee_id()
```

```
get_employee_name()
```

```
is_valid_employee()
```

```
find_employee_by_id()
```

`get_valid_employees()`

`add_employee()`

`remove_employee()`

`update_employee()`

`find_employee()`

`list_employees()`

`search_employees()`

`load_data()`

`save_data()`

`display_menu()`

`get_menu_choice()`

`handle_menu_choice()`

`run_application()`

`setup_logging()`

The names make it clear what each function does.

For example:

`save_data()`

is easier to understand than a name such as:

`process()`

15. Parameters

Parameters are used when a function needs some information from outside.

For example:

```
def get_employee_name(message):
```

The same function can be used for different messages:

```
get_employee_name("Enter Employee Name: ")
```

and:

```
get_employee_name("Enter New Employee Name: ")
```

Another example:

```
def find_employee_by_id(employee_id):
```

The employee ID is passed to the function so it knows which employee to search for.

16. Return Values

Functions return values when another part of the program needs the result.

For example:

```
def find_employee_by_id(employee_id):
```

returns the employee if it is found:

```
return employee
```

If the employee is not found:

```
return None
```

The `is_valid_employee()` function returns:

True

when the employee is valid and:

False

when it is invalid.

This makes these functions reusable.

17. Reusable Functions

Instead of writing the same logic multiple times, common logic is placed inside reusable functions.

For example:

```
find_employee_by_id()
```

is used when:

- Adding an employee
- Removing an employee
- Updating an employee

- Finding an employee

Another example is:

`get_valid_employees()`

which is used by:

- `list_employees()`
- `search_employees()`

This avoids repeating the same employee-validation loop.

18. Separation of Responsibilities

The project separates different types of work.

`validation.py`

↓

Input Validation

`employee.py`

↓

Employee Operations

`main.py`

↓

Application Flow + JSON + Logging

This makes the project easier to understand because we know where to look for a particular piece of logic.

For example, if there is a problem with employee ID validation, we can check `validation.py` instead of searching through the complete project.

19. Challenge – Three Areas Improved

Improvement 1 – Employee Search

Before

Employee search logic could be written directly inside different operations:

for employee in employees:

 if employee["id"] == employee_id:

 ...

Why It Is a Problem

If the same logic is written in multiple functions, the same code gets repeated.

If we need to change the search logic later, we may have to change it in multiple places.

After

A separate reusable function was created:

```
def find_employee_by_id(employee_id):
```

```
    for employee in employees:
```

```
        if not is_valid_employee(employee):
```

```
            logger.error("Invalid employee data found: %s", employee)
```

```
            print("Invalid employee data found.")
```

```
            continue
```

```
        if employee["id"] == employee_id:
```

```
            return employee
```

```
    return None
```

Now other functions can simply use:

```
employee = find_employee_by_id(employee_id)
```

Why It Is Better

The search logic is written only once.

It is easier to read, reuse, and maintain.

20. Improvement 2 – Employee Validation

Before

Employee validation could be written separately wherever it was needed.

For example:

```
if not isinstance(employee, dict):
```

```
    ...
```

```
if "id" not in employee:
```

```
    ...
```

```
if "name" not in employee:
```

```
    ...
```

Why It Is a Problem

The same validation rules could be repeated in multiple places.

This makes the code longer and harder to maintain.

After

A separate function was created:

```
def is_valid_employee(employee):
```

```
    if not isinstance(employee, dict):
```

```
        return False
```

```
    if "id" not in employee or "name" not in employee:
```

```
        return False
```

```
    if not isinstance(employee["id"], int):
```

```
        return False
```

```
    if employee["id"] <= 0:
```

```
        return False
```

```
if not isinstance(employee["name"], str):
```

```
    return False
```

```
if employee["name"].strip() == "":
```

```
    return False
```

```
    return True
```

Why It Is Better

Now the employee validation rules are in one place.

The same function can be used whenever employee data needs to be checked.

This makes the code easier to understand and test.

21. Improvement 3 – Separate Menu Handling

Before

The application could become difficult to read if menu display, user input, operation selection, and application flow were all handled together.

Why It Is a Problem

Too many responsibilities inside one function make the function difficult to understand.

It also becomes harder to find where a particular problem is happening.

After

The responsibilities were separated into different functions:

```
display_menu()
```

Displays the menu.

```
get_menu_choice()
```

Gets the user's choice.

```
handle_menu_choice(choice)
```

Calls the correct operation.

```
run_application()
```

Controls the application loop.

main()

Starts the application and handles top-level errors.

Why It Is Better

Each function has a simple job.

The flow becomes:

main()

↓

run_application()

↓

display_menu()

↓

get_menu_choice()

↓

handle_menu_choice()

↓

Employee Operation

This makes the code easier to follow.